



University of Ioannina

Department of Computer Science and Engineering

Documentation for Franka Research 3

Force Control and Human–Robot Interaction

Author: Danai Chanlaridou

Academic Year: 2025–2026

Document Type: Technical Documentation

Ioannina, October 2026

Contents

1	System Overview	1
1.1	Complete System Architecture	1
1.2	When to Use Each Environment	1
1.2.1	Direct libfranka	1
1.2.2	ROS 2 and MoveIt	1
1.2.3	Gazebo	2
2	Hardware Setup	3
2.1	Franka Research 3 Arm	3
2.2	Integrated Torque Sensing	3
2.3	Franka Hand	4
2.4	Workstation	4
2.5	Detailed Hardware Configuration and Operating Limits	4
2.5.1	FR3 Arm Hardware Characteristics	4
2.5.2	Franka Hand Specifications and End-Effector Payload	5
2.5.3	Mechanical Mounting and Installation Requirements	6
2.5.4	Joint Position Limits	7
2.5.5	Joint Velocity and Cartesian Speed Limits	8
2.5.6	Control Unit, Interfaces, and Network Configuration	8
2.5.7	Real-Time Communication and FCI Constraints	9
2.5.8	Safety Functions and Watchman Configuration	9
2.5.9	Practical Limits Used for Thesis Experiments	10
3	Software Setup	12
3.1	Main Workspaces	12
3.1.1	franka_ros2 workspace	12
3.1.2	Direct non-ROS scripts	12
3.1.3	Force-control simulation workspace	13
4	Network Architecture and Desk Access	14
4.1	Important Network Architecture	14
4.2	Correct Physical Connection	14
4.3	Linux Static IP Configuration for Controller Access	15
4.4	Verifying Physical Ethernet Link	15
4.5	Why 192.168.0.1 May Not Work	15
4.6	Optional X5 Direct Connection	16

5	Real-Time Linux and Permissions	17
5.1	Why Real-Time Configuration Matters	17
5.2	Useful Real-Time Checks	17
5.3	Running Programs With Correct libfranka Path	17
6	Direct FR3 Control Without ROS Using libfranka	19
6.1	Purpose of the Direct libfranka Approach	19
6.2	Typical C++ Includes	19
6.3	Robot and Gripper Connection	19
6.4	Communication Test With echo_robot_state	20
6.5	Network Checks for Direct Control	20
6.6	Common libfranka Communication Problem	20
7	FR3 Control With ROS 2 and MoveIt	22
7.1	Purpose of ROS 2 Control	22
7.2	Sourcing ROS Environments	22
7.3	Real-Robot MoveIt Launch	22
7.4	Successful ROS Connection Indicators	23
7.5	Direct libfranka vs ROS 2	23
8	Gazebo Simulation Environment	24
8.1	Purpose of Gazebo in the Project	24
8.2	Why Simulation Is Useful but Limited	24
8.3	Typical Gazebo Issues	25
8.4	Modules Needed for FR3 Simulation	25
8.4.1	URDF/Xacro	25
8.4.2	ros2_control	25
8.4.3	Gazebo Plugins	25
9	Coordinate Frames and Homogeneous Transformations	26
9.1	Pose Representation	26
9.2	Implemented Transform Function	26
9.3	Top-Down Grasp Orientation	26
9.4	Why the Correct Frame Matters	27
10	Smooth Cartesian Motion Generation	28
10.1	Point-to-Point Cartesian Motion	28
10.2	Quintic Smoothstep Interpolation	28
10.3	Why Smooth Motion Is Important	28
10.4	Orientation Interpolation	29
11	Force and Torque Measurements	30
11.1	External Wrench Estimate	30
11.2	External Joint Torque Estimate	30
11.3	Why Force/Torque Logging Is Important	30

12 Implemented Force-Control Methods	31
12.1 Impedance Control	31
12.1.1 Concept	31
12.1.2 Torque Command	31
12.1.3 Practical Meaning	32
12.2 Admittance Control	32
12.2.1 Concept	32
12.2.2 Practical Meaning	32
12.3 Hybrid Force/Position Control	33
12.3.1 Concept	33
12.3.2 Cartesian Wrench	33
12.3.3 Why Hybrid Control Was Needed	33
13 Force Bias Compensation	34
13.1 Why Bias Compensation Is Needed	34
13.2 Bias Estimation	34
13.3 Force Sign Used in the Experiments	34
14 Dice Surface Transfer Experiment	36
14.1 Experiment Objective	36
14.2 Task Sequence	36
14.3 Important Experiment Values	36
14.4 Why Calibrated End-Effector z Values Were Used	37
15 Additional ROS 2 and MoveIt Debugging Notes	38
15.1 RViz and MoveIt Debugging	38
15.2 Custom ROS 2 Package Structure	39
16 Practical Commands Used During the Experimental Workflow	40
16.1 Basic ROS 2 and Workspace Setup	40
16.2 Ethernet Configuration for the FR3 Controller	40
16.3 Alternative Network Configuration Using nmcli	41
16.4 Desk Preparation Before Real Robot Control	42
16.5 Testing libfranka Communication	42
16.6 Building Direct libfranka C++ Programs	43
16.7 Running Direct libfranka Programs	43
16.8 Clean Build of the ROS 2 Franka Workspace	44
16.9 Running MoveIt with Fake Hardware	45
16.10 Checking MoveIt Planning Groups	45
16.11 Temporary RViz Semantic Relay Fix	46
16.12 Running Gazebo Simulation	48
16.13 Running the Real FR3 with MoveIt	48
16.14 Running Real FR3 Example Controllers	49
16.15 Monitoring During ROS Experiments	50
16.16 Final Command Sequence Before a Real Force-Control Experiment	50
16.17 Summary of When Each Command Group Is Used	51
16.18 Final Reproducible Workflow	52

17 Troubleshooting Guide	53
17.1 Desk Opens but the Arm Is Disconnected	53
17.2 Recovery Procedure for Arm Disconnection	53
17.3 Link detected: no	54
17.4 libfranka: UDP receive: Timeout	54
17.5 Desk Reachable but 192.168.0.1 Unreachable	55
17.6 X3.1 Triggered	55
17.7 Self Test Fails With Internal Error	55
17.8 WATCHMAN Shows Validated but Robot Is Not Operational	55
17.9 Login Mode	55
17.10 Gripper Does Not Confirm Grasp	56
17.11 Force Does Not Increase	56
17.12 Force Is Too High or Unstable	56
17.13 Final Position Error Is Too Large	57

Chapter 1

System Overview

1.1 Complete System Architecture

The project uses two main real-robot control routes:

1. **Direct libfranka control without ROS**

This is the low-level route. It is useful for custom torque control, impedance control, admittance control, force/torque reading, and strict real-time experiments.

2. **ROS 2 / MoveIt control**

This is the higher-level route. It is useful for visualization, planning, testing standard controllers, RViz monitoring, and integration with larger robotic systems.

Gazebo simulation is used as a third environment. It is not a replacement for real contact testing, but it is extremely useful for testing geometry, launch files, object placement, and approximate robot behavior before using the real robot.

1.2 When to Use Each Environment

1.2.1 Direct libfranka

Use direct libfranka when the experiment requires:

- direct robot-state access
- direct torque commands
- direct model access
- 1 kHz control callbacks
- custom impedance/admittance/hybrid force control
- CSV logging from low-level state variables

1.2.2 ROS 2 and MoveIt

Use ROS 2 and MoveIt when the experiment requires:

- visualization in RViz
- motion planning
- standard trajectory controllers
- planning scene management
- integration with ROS nodes
- higher-level robot application development

1.2.3 Gazebo

Use Gazebo when the experiment requires:

- testing the scene before using the real robot
- checking table, wall, or object placement
- validating approximate reachability
- debugging URDF/Xacro and launch files
- simulating object interaction before hardware deployment

Chapter 2

Hardware Setup

The hardware setup consists of:

- Franka Research 3 robotic arm
- Franka controller / black control unit / industrial PC
- Franka Hand gripper
- Ubuntu workstation
- Ethernet cable for controller communication
- Desk browser interface for activation and recovery
- Table used for physical experiments
- Test objects, mainly a dice

2.1 Franka Research 3 Arm

The FR3 is a 7-degree-of-freedom collaborative manipulator. Its 7-DOF structure is important because it gives the robot redundancy. For a normal 6D end-effector pose, six degrees of freedom are sufficient. The seventh degree of freedom allows the robot to change its elbow posture while keeping the same end-effector pose.

This is useful for:

- obstacle avoidance
- avoiding joint limits
- improving manipulability
- keeping a safer posture during human interaction
- avoiding singularities

2.2 Integrated Torque Sensing

A major advantage of the FR3 is that it provides joint torque sensing. This makes the robot appropriate for force-control research because interaction forces can be estimated through the robot model and torque residuals.

This reduces the need for a dedicated external wrist force/torque sensor for many experiments.

2.3 Franka Hand

The Franka Hand was used for object interaction. In the final experiment, it was used to grasp a dice before performing a surface transfer motion.

Important gripper-related parameters used in the experiments include:

```
dice_width_m = 0.040
gripper_force_N = 25.0
```

The expected successful gripper message was:

```
Gripper grasp confirmed.
```

2.4 Workstation

The workstation was an Ubuntu Linux machine configured for real-time robot communication. For real-time force control, the operating system and user permissions are critical because the robot expects a stable communication loop.

2.5 Detailed Hardware Configuration and Operating Limits

This section summarizes the main hardware characteristics, configuration parameters, and operating limits that were relevant for the experiments of this thesis. The values are based on the manufacturer documentation for the Franka Research 3 arm and the Franka Hand. They are included here because force-control experiments depend strongly on the physical limits of the robot, the configuration of the end effector, the network setup, and the safety functions active in the controller.

2.5.1 FR3 Arm Hardware Characteristics

The Franka Research 3 arm is a seven-degree-of-freedom collaborative manipulator. The seven revolute joints provide kinematic redundancy, which is useful for posture optimization, obstacle avoidance, and singularity avoidance while keeping the end-effector task unchanged. The main hardware specifications used during the design of the experiments are summarized in Table 2.1.

Table 2.1: Main FR3 arm specifications relevant to the thesis experiments.

Quantity	Value / Description
Degrees of freedom	7 revolute joints
Maximum payload	3 kg, including end effector, object, and any additional mounted equipment
Maximum reach	855 mm
Arm weight	Approximately 17.8 kg
Force/torque sensing	Link-side torque sensor in all seven axes
Mounting flange	DIN ISO 9409-1-A50

Quantity	Value / Description
Installation position	Upright installation only
Protection rating	IP40 for the Arm
Ambient temperature	+5 °C to +45 °C extended operating range
Relative humidity	20–80%, non-condensing
Pose repeatability	Less than ± 0.1 mm under the specified reference conditions
Guiding force	Approximately 2.5 N
Adjustable translational stiffness	10–3000 N/m
Adjustable rotational stiffness	1–300 Nm/rad
Monitored signals	Joint position, joint velocity, joint torque, Cartesian position, and force

These specifications influence the controller design directly. The payload limit defines how much mass can be mounted at the flange, including the gripper and the grasped object. The integrated joint torque sensing is especially important for this thesis, because it allows interaction forces to be estimated without requiring an additional wrist-mounted force/torque sensor for every experiment. The torque sensing and the 1 kHz Franka Control Interface make the FR3 suitable for torque-based impedance control, admittance experiments, and hybrid force/position control.

2.5.2 Franka Hand Specifications and End-Effector Payload

The Franka Hand is an electrical two-finger parallel gripper. It is mounted directly on the robot flange and does not require external cabling, because power and communication are provided through the X6 end-effector connector. For the experiments of this thesis, the Franka Hand was used for grasping small objects before performing surface-contact manipulation.

Table 2.2: Main Franka Hand specifications relevant to the experiments.

Quantity	Value / Description
Type	Electrical two-finger parallel gripper
Outer dimensions	Width 63 mm, length 205 mm, height 127 mm
Weight	730 g
Center of mass relative to flange	$x = 0$ mm, $y = 9.3$ mm, $z = 30.2$ mm
Continuous grasping force	Adjustable, 30–70 N
Travel range	80 mm
Travel speed	50 mm/s per finger
Mechanical interface	DIN ISO 9409-1-A50 compatible flange interface
Electrical interface	X6 end-effector connector at the robot flange
Protection rating	IP20 for the Hand

The mass and center of mass of the end effector must be configured correctly in the Franka user interface. Incorrect end-effector mass or center-of-mass values can lead to

wrong gravity compensation, unexpected motion during guiding, reduced sensitivity of collision detection, and degraded tracking behavior. Therefore, when the Franka Hand is mounted, its mass and center of mass must be included in the end-effector configuration. If an object is grasped and its mass is significant, the total payload model should also include the object.

For the final object-transfer experiments, the payload condition can be written as:

$$m_{payload} = m_{hand} + m_{object} + m_{additional\ equipment} \quad (2.1)$$

and the following condition must remain satisfied:

$$m_{payload} \leq 3 \text{ kg}. \quad (2.2)$$

Because the Franka Hand has a mass of approximately 0.73 kg, the remaining available payload for the grasped object and any additional tooling is approximately:

$$m_{remaining} \approx 3.00 - 0.73 = 2.27 \text{ kg}. \quad (2.3)$$

In the experiments of this thesis, the manipulated objects were lightweight, so the payload limit was not the critical limitation. However, the end-effector configuration was still important for gravity compensation, force estimation, and safety monitoring.

2.5.3 Mechanical Mounting and Installation Requirements

The FR3 must be installed on a stable, level, and non-moving platform. This is especially important for force-control experiments because vibrations or base motion can affect torque estimates and measured contact forces. The arm is designed for upright mounting, with the base horizontal to the ground. Hanging or tilted installation is not allowed for this setup.

The installation site must satisfy the following practical conditions:

- indoor installation in an enclosed building,
- no direct sunlight on the robot,
- no vibrations or accelerating base/platform,
- no flammable, aggressive, corrosive, or spraying substances,
- no pressurized air streams acting on the robot,
- pollution degree 2 environment,
- altitude not greater than 2000 m above sea level,
- ambient temperature within the specified operating range,
- relative humidity between 20% and 80%, non-condensing.

The base platform must support the static and dynamic loads generated by the robot. The manufacturer specifies the following maximum loads at the mounting base:

Table 2.3: Maximum mechanical loads to be supported by the FR3 mounting platform.

Load type	Maximum value
Tilting torque	280 Nm
Torque around the vertical axis	190 Nm
Horizontal force	300 N
Vertical force	410 N
Maximum permitted platform tilt	0.1°

The arm is mounted using four M8 screws. The tightening torque for the base screws is 30 Nm. The screws should be checked after approximately 100 hours of operation. The functional earth connection must also be connected according to the manufacturer instructions in order to meet the stated EMC levels.

The Franka Hand is mounted to the end-effector flange using two M6x12 DIN 7984 screws with 5 Nm tightening torque. The Hand plug must be inserted into the X6 connector on the robot flange. The hand must not be connected or disconnected while the robot system is powered.

2.5.4 Joint Position Limits

The software trajectories and controller commands must always remain inside the mechanical and safety limits of the robot. The FR3 joint position limits are summarized in Table 2.4. These values are important when designing joint-space trajectories, null-space posture control, and safety checks for inverse kinematics.

Table 2.4: FR3 joint position limits.

Joint	Lower limit	Upper limit
A1	−166°	166°
A2	−105°	105°
A3	−166°	166°
A4	−176°	−7°
A5	−165°	165°
A6	25°	265°
A7	−175°	175°

In the implementation, these limits were treated as hard physical constraints. For safe experiments, the commanded trajectories should not only remain inside the limits, but also maintain a margin from them. A practical joint-limit margin can be expressed as:

$$q_{i,min} + \Delta q_{safe} \leq q_i \leq q_{i,max} - \Delta q_{safe}, \quad (2.4)$$

where Δq_{safe} is a selected safety margin. This is useful because force-control experiments can introduce additional motion due to contact, compliance, or external disturbances.

2.5.5 Joint Velocity and Cartesian Speed Limits

The velocity limits define the maximum allowed joint and Cartesian motion speeds. They are important for trajectory design, impedance experiments, admittance experiments, and emergency stopping considerations. For safe laboratory experiments, the implemented velocities should be significantly lower than the maximum allowed values.

Table 2.5: FR3 velocity and motion limits relevant to the experiments.

Quantity	Limit
Joint velocity, A1–A4	150°/s
Joint velocity, A5–A7	301°/s
Joint velocity, A6 when using FCI	239°/s
Cartesian end-effector velocity	Up to 2 m/s
Franka Hand finger speed	50 mm/s per finger

Although the maximum Cartesian end-effector speed is specified as up to 2 m/s, the force-control experiments in this thesis require much lower speeds. Low speed is important because contact force depends on the dynamic interaction between the robot, the object, and the environment. A conservative commanded velocity reduces impact forces, improves stability, and gives the safety functions more time to react.

A practical software rule for safe motion generation is:

$$|\dot{q}_{cmd,i}| \leq \dot{q}_{i,safe}, \quad \dot{q}_{i,safe} < \dot{q}_{i,max}, \quad (2.5)$$

and for Cartesian motion:

$$\|v_{cmd}\| \leq v_{safe}, \quad v_{safe} < v_{max}. \quad (2.6)$$

In other words, the robot software should use internal limits that are more conservative than the hardware limits. This is especially important when the controller is tested on the real FR3 after simulation.

2.5.6 Control Unit, Interfaces, and Network Configuration

The FR3 system consists of the arm and the Control unit. The Control unit is the main controller of the robot and provides the shop-floor network connection used for FCI experiments. The operation of the Arm is permitted only with the Control provided by Franka Robotics.

The main interfaces used in this thesis were:

- **C2 – Shop-floor network:** Ethernet interface on the Control unit. This is the required interface for Franka Control Interface experiments.
- **X5 – Robot network:** Ethernet interface on the robot base. This is used for Franka UI/Desk access on the robot-side network, but it is not suitable for FCI control.
- **X6 – End-effector connector:** electrical interface at the wrist/flange used by the Franka Hand.

- **X3 – Safe inputs:** interface for the emergency stop and configurable safe inputs.
- **X4 – External enabling:** interface for the external enabling device.

For FCI operation, the workstation must be connected to the C2 shop-floor network port of the Control. In the experimental setup, the static network configuration was:

Table 2.6: Static network configuration used for FCI experiments.

Device	IP address	Netmask
Ubuntu workstation	172.16.0.1	/24
FR3 Control / FCI address	172.16.0.2	/24
Robot-side X5 network	192.168.0.1	/24

The C2 connection is the correct route for real-time FCI control. The X5 robot network can be useful for Desk access and initial configuration, but it must not be used as the real-time FCI control connection. Before any real experiment, the controller connection must be verified by ping and then by reading the robot state through `libfranka`.

2.5.7 Real-Time Communication and FCI Constraints

The Franka Control Interface provides a 1 kHz research interface. This means that the external workstation must exchange data with the robot controller at a stable real-time rate. For this reason, network stability and real-time Linux configuration are part of the hardware setup, not only software details.

The workstation should use:

- a Linux operating system suitable for real-time communication,
- a real-time or low-latency kernel configuration,
- stable wired Ethernet connection directly to the Control unit,
- correct real-time permissions for the user running the controller,
- no competing robot-control process connected to the robot.

The real-time callback must remain computationally simple. Expensive operations, file handling, plotting, or blocking operations should not be performed inside the real-time control loop. For this reason, the implemented controllers compute only the required control law inside the callback, while logging is kept simple and deterministic.

2.5.8 Safety Functions and Watchman Configuration

The FR3 provides several safety functions through its safety system and the Watchman interface. These functions are essential for defining safe robot behavior in the laboratory. The most relevant safety-related concepts for this thesis are summarized below.

Table 2.7: Relevant FR3 safety functions and practical meaning.

Function	Practical meaning
Emergency Stop, X3.1	Triggers a safety stop through the emergency stop device.
External Enabling, X4	Enables motion in modes where an external enabling device is required.
Enabling Button	Three-position enabling device on the Pilot-Grip for guiding and collaborative operation.
SLP-C	Safely limited Cartesian position; monitors Cartesian position of selected robot/tool points.
SLS-C	Safely limited Cartesian speed; monitors Cartesian speed of selected robot/tool points.
SLP-J	Safely limited joint angle; internally protects joint limits and self-collision constraints.
SLS-J	Safely limited joint speed; monitors joint speed.
SMSS	Safely monitored standstill.
SEEPS	Safe End Effector Power Off; switches off 48 V end-effector power according to the configured safety behavior.

A key practical point is that some safety functions can prevent FCI control when they are active. Therefore, the robot must be in a valid operating state, the safety configuration must be validated, and the required safety violations must be recovered before running a `libfranka` controller.

When the Franka Hand or any other tool is mounted, the end-effector geometry must also be represented in Watchman. The safety system can model the end effector using spheres attached to the flange. These spheres are used for spatial and speed monitoring. If a long object is grasped, the sphere model should cover the object as well, because the maximum speed is monitored at the sphere centers. This is particularly important in surface-contact experiments, where the gripper and the object may extend the effective robot geometry.

2.5.9 Practical Limits Used for Thesis Experiments

The manufacturer limits define the maximum allowed operating envelope. The experimental limits used in a thesis or laboratory environment should be more conservative. The reason is that force-control experiments involve contact, uncertainty, friction, sensor bias, and possible human interaction.

For the experiments of this thesis, the following practical principles were applied:

- start with very small joint or Cartesian motions,
- use low Cartesian speeds during contact approach,
- keep the robot away from joint limits,
- avoid fully stretched configurations and singular postures,
- use low initial force targets and increase only after validation,

- apply torque-rate saturation in torque controllers,
- apply force limits and vertical safety walls during contact tasks,
- keep the emergency stop accessible during all real robot experiments,
- verify Desk, FCI activation, ping, and `echo_robot_state` before motion,
- stop immediately if abnormal forces, unexpected motion, or safety warnings appear.

This conservative approach is necessary because the real robot is not only a mathematical model. Its behavior depends on payload configuration, friction, table height, object geometry, force bias, controller gains, and safety-state configuration. Therefore, every real experiment must start from a verified and low-energy condition before moving to more complex force-control tasks.

Chapter 3

Software Setup

The software stack used in the project includes:

```
Ubuntu Linux
ROS 2 Humble
libfranka
franka_ros2
MoveIt 2
Gazebo / Ignition Gazebo
Eigen C++
CMake
colcon
```

Important directories used during development:

```
~/franka_ros2_ws
~/Desktop/scripts_no_ros
~/Desktop/fr3_force_control_ws
```

3.1 Main Workspaces

3.1.1 franka_ros2 workspace

```
~/franka_ros2_ws
```

This workspace contains the Franka ROS 2 packages, MoveIt configuration, robot descriptions, hardware interfaces, and the installed libfranka build.

3.1.2 Direct non-ROS scripts

```
~/Desktop/scripts_no_ros
```

This folder contains standalone C++ programs that communicate directly with the real robot using libfranka.

Example programs:

```
fr3_pick_direct
fr3_pick_place_ball
fr3_read_forces
fr3_impedance_task_test
```

```
fr3_admittance_task_test  
fr3_hybrid_force_position_test  
fr3_dice_surface_transfer_force
```

3.1.3 Force-control simulation workspace

```
~/Desktop/fr3_force_control_ws
```

This workspace was used for Gazebo/Ignition Gazebo simulation experiments.

Chapter 4

Network Architecture and Desk Access

4.1 Important Network Architecture

The FR3 laboratory setup uses two different networks:

Network	Purpose	IP
Controller / Shop Floor Network	Access to Desk UI and controller	172.16.0.2
Robot / X5 Network	Internal robot-side network	192.168.0.1

This distinction is important because both addresses may appear in documentation, but they do not always behave the same way.

The main IP used during the real robot experiments was:

172.16.0.2

Most programs were executed in the following form:

```
./program_name 172.16.0.2
```

4.2 Correct Physical Connection

For normal controller access, the laptop Ethernet cable should be connected to the:

`black controller box / industrial PC`

and **not directly to the robot base**, unless a specific X5 direct connection is being tested.

Desk should be opened through:

`https://172.16.0.2`

The HTTPS certificate warning is normal.

4.3 Linux Static IP Configuration for Controller Access

First identify the Ethernet interface:

```
ip addr
```

In the lab setup, the Ethernet interface was usually:

```
enp3s0
```

Configure the laptop Ethernet interface:

```
sudo ip addr flush dev enp3s0
sudo ip addr add 172.16.0.1/24 dev enp3s0
sudo ip link set enp3s0 up
```

Then test connectivity:

```
ping 172.16.0.2
```

If ping works, the controller network is reachable.

4.4 Verifying Physical Ethernet Link

Use:

```
sudo ethtool enp3s0
```

Expected result:

```
Link detected: yes
```

If the result is:

```
Link detected: no
```

then the problem is physical. Possible causes include:

- disconnected cable
- wrong port
- cable not clicked properly
- controller powered off
- damaged cable

4.5 Why 192.168.0.1 May Not Work

The 192.168.0.1 X5 network only becomes active when the robot arm has initialized correctly and the controller has connected to it.

If Desk shows:

```
Arm disconnected
Safety booting...
No Arm Connected
```

then 192.168.0.1 may not respond at all.

This does not necessarily mean that the laptop network setup is wrong. It may mean that the controller is running but the arm subsystem has not booted or is not connected.

4.6 Optional X5 Direct Connection

After the arm initializes correctly, the X5 network can be tested directly.

Configure the laptop as:

```
sudo ip addr flush dev enp3s0
sudo ip addr add 192.168.0.2/24 dev enp3s0
sudo ip link set enp3s0 up
```

Test:

```
ping 192.168.0.1
```

Open Desk:

```
https://192.168.0.1
```

However, for the main experiments, the controller access through 172.16.0.2 was the recommended setup.

Chapter 5

Real-Time Linux and Permissions

5.1 Why Real-Time Configuration Matters

The FR3 control loop requires stable, low-latency communication. In torque-control experiments, the robot expects commands at high frequency. If the control loop misses timing deadlines, the robot may stop for safety reasons.

This is especially important for:

- impedance control
- admittance control
- hybrid force/position control
- torque-level experiments
- smooth Cartesian servoing

5.2 Useful Real-Time Checks

Check real-time priority:

```
ulimit -r
```

Expected:

```
99
```

Check memory locking:

```
ulimit -l
```

Expected:

```
unlimited
```

High real-time priority and unlimited memory locking help prevent timing instability in the control loop.

5.3 Running Programs With Correct libfranka Path

Many programs were executed using `sudo` and an explicit `LD_LIBRARY_PATH`:

```
sudo env
    LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH
    \
./program_name 172.16.0.2
```

This command does two important things:

1. It runs the program with the permissions needed for reliable robot control.
2. It forces the program to use the correct libfranka shared library from the workspace installation.

This avoids situations where the system accidentally loads a different or incompatible libfranka version.

Chapter 6

Direct FR3 Control Without ROS Using libfranka

6.1 Purpose of the Direct libfranka Approach

The direct non-ROS implementation communicates with the robot using libfranka only. It does not depend on ROS nodes, ROS controllers, MoveIt, or Gazebo.

This approach is the most appropriate for low-level experiments because it provides direct access to:

- robot state
- joint positions and velocities
- measured torques
- external torque estimates
- Cartesian pose
- robot dynamic model
- Jacobian
- Coriolis terms
- gravity compensation
- torque command callback
- Cartesian pose command callback

6.2 Typical C++ Includes

A typical direct-control C++ program includes:

```
#include <franka/robot.h>
#include <franka/gripper.h>
#include <franka/model.h>
#include <franka/control_types.h>
```

Eigen is also commonly used for vector and matrix operations:

```
#include <Eigen/Dense>
```

6.3 Robot and Gripper Connection

Robot connection:


```
franka::Robot robot(robot_ip);

    Gripper connection:
franka::Gripper gripper(robot_ip);

    Single state read:
franka::RobotState state = robot.readOnce();

    Real-time control callback:
robot.control(callback);
```

6.4 Communication Test With echo_robot_state

A basic communication test is:

```
sudo env
    LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH
    \
/home/danai/franka_ros2_ws/install/libfranka/bin/echo_robot_state 172.16.0.2
```

If communication works, the robot state is printed continuously.

This test verifies more than simple network ping. Ping only checks basic IP connectivity, while `echo_robot_state` verifies that libfranka can receive real robot-state packets.

6.5 Network Checks for Direct Control

Useful commands:

```
ping 172.16.0.2
ip route get 172.16.0.2
sudo ufw status
sudo iptables -L -n -v
```

Important distinction:

A successful ping does not guarantee that libfranka communication works.

libfranka requires stable robot-state packet reception and correct FCI activation.

6.6 Common libfranka Communication Problem

A common error is:

```
libfranka: UDP receive: Timeout
```

Meaning:

The PC may be able to ping the controller, but libfranka is not receiving the required robot-state packets.

Possible causes:

- FCI is not activated
- the robot is in an error or reflex state
- another process is still connected to the robot
- firewall or UDP issue
- wrong network route
- unstable real-time communication
- Desk did not complete recovery

Useful cleanup commands:

```
sudo pkill -f fr3_pick_place_ball
sudo pkill -f fr3_pick_direct
sudo pkill -f fr3_admittance_task_test
sudo pkill -f fr3_impedance_task_test
sudo pkill -f echo_robot_state
```

After killing previous processes, the robot should be recovered in Desk and FCI should be activated again.

Chapter 7

FR3 Control With ROS 2 and MoveIt

7.1 Purpose of ROS 2 Control

The ROS 2 approach was used for higher-level robot control and visualization.

ROS 2 is useful for:

- motion planning
- RViz visualization
- MoveIt integration
- planning scene management
- using standard controllers
- debugging robot model and robot state
- integrating external sensors or higher-level logic

7.2 Sourcing ROS Environments

Before running ROS commands:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash
```

The first command loads the system ROS 2 Humble installation. The second command loads the local Franka workspace.

7.3 Real-Robot MoveIt Launch

Typical launch command:

```
ros2 launch franka_fr3_moveit_config moveit.launch.py \
robot_ip:=172.16.0.2 \
arm_id:=fr3 \
use_fake_hardware:=false
```

This starts:

- robot_state_publisher
- ros2_control_node
- FrankaHardwareInterface
- controller spawners

- MoveIt move_group
- RViz
- Franka gripper node

7.4 Successful ROS Connection Indicators

Successful logs include:

```
robot model 'fr3' loaded
FrankaHardwareInterface successfully connected to robot
ros2_control command interfaces registered
grripper connected
MoveIt planning scene initialized
```

If these messages appear, ROS 2 has successfully connected to the real robot.

7.5 Direct libfranka vs ROS 2

The two approaches should not be treated as identical.

Direct libfranka is better for:

- custom real-time control
- torque-level experiments
- force-control development
- standalone experiments

ROS 2 and MoveIt are better for:

- planning and visualization
- standard robot applications
- RViz monitoring
- integration with a larger robotics system

A practical rule is:

Use direct libfranka for the control algorithm itself.

Use ROS 2/MoveIt when planning, visualization, or integration is more important.

Chapter 8

Gazebo Simulation Environment

8.1 Purpose of Gazebo in the Project

Gazebo / Ignition Gazebo was used to test robot scenes and motions before running them on the real hardware.

The simulation workspace was:

```
~/Desktop/fr3_force_control_ws
```

Gazebo was useful for:

- testing the visual scene
- placing tables, objects, and walls
- checking approximate reachability
- debugging launch files
- experimenting with robot/environment geometry
- testing workflows before using the real robot

8.2 Why Simulation Is Useful but Limited

Gazebo is useful because it can simulate:

- gravity
- collisions
- approximate rigid-body dynamics
- joint motion
- object placement
- contact reactions

However, simulation does not perfectly reproduce real contact behavior. Friction, contact stiffness, object compliance, gripper contact, table surface properties, and sensor noise may differ from reality.

Therefore:

Gazebo is reliable for workflow, geometry, and approximate behavior.
Real robot experiments are required for validating actual force-control performance.

8.3 Typical Gazebo Issues

Common simulation issues included:

- missing ROS/Gazebo packages
- Gazebo closing automatically
- object, table, or wall placed incorrectly
- table height not matching expected task height
- gripper not visually reaching the objects
- wall overlapping the robot

These issues were solved by adjusting world files, object positions, table height, and launch configuration.

8.4 Modules Needed for FR3 Simulation

To simulate the FR3, the following components are typically used:

URDF/Xacro
ros_control / ros2_control
Gazebo plugins
robot description meshes
joint controllers

8.4.1 URDF/Xacro

URDF/Xacro describes:

- robot kinematics
- visual meshes
- collision geometry
- link inertias
- joint limits
- coordinate frames

8.4.2 ros2__control

ros2_control acts as the bridge between controllers and the simulated or real robot hardware.

For force-control-related simulation, effort interfaces are important because they allow torque-like commands instead of only position commands.

8.4.3 Gazebo Plugins

Gazebo plugins can simulate:

- robot hardware interfaces
- cameras
- depth sensors
- contact sensors
- force/torque sensors

Chapter 9

Coordinate Frames and Homogeneous Transformations

9.1 Pose Representation

Robot poses are represented using homogeneous transformation matrices:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

where:

$$p = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

and R is the rotation matrix.

The vector p describes the Cartesian position of a frame, while R describes its orientation.

9.2 Implemented Transform Function

The implemented function:

```
makeTransform(x, y, z, roll, pitch, yaw)
```

uses the rotation convention:

$$R = R_z(yaw)R_y(pitch)R_x(roll)$$

This means that roll is applied around the x-axis, pitch around the y-axis, and yaw around the z-axis, combined in the above order.

9.3 Top-Down Grasp Orientation

For top-down grasping, the following orientation was used:

```
roll = pi;  
pitch = 0;  
yaw = 0;
```

This gives approximately:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

This orientation points the end-effector downward toward the table, which is appropriate for grasping objects from above.

9.4 Why the Correct Frame Matters

In robot control, confusing frames is a common source of errors.

For example:

- a table height measured in the world frame is not necessarily the commanded end-effector z value,
- a gripper fingertip position differs from the end-effector frame,
- a tool frame may be offset from the flange frame,
- a force sign may appear inverted depending on the frame convention.

This is why the final experiment used calibrated end-effector z values instead of relying only on the measured physical table height.

Chapter 10

Smooth Cartesian Motion Generation

10.1 Point-to-Point Cartesian Motion

For point-to-point Cartesian motion, the robot interpolates between an initial position p_0 and a target position p_1 :

$$p(t) = p_0 + \alpha(t)(p_1 - p_0)$$

where p_0 is the starting position, p_1 is the target position, and $\alpha(t)$ is a smooth interpolation factor.

10.2 Quintic Smoothstep Interpolation

The interpolation function is:

$$\alpha(s) = 10s^3 - 15s^4 + 6s^5$$

where:

$$s = \frac{t}{T}$$

and T is the total movement duration.

This function has the useful properties:

$$\dot{\alpha}(0) = \dot{\alpha}(1) = 0$$

$$\ddot{\alpha}(0) = \ddot{\alpha}(1) = 0$$

This means that the movement starts and ends smoothly, with zero velocity and zero acceleration at the boundaries.

10.3 Why Smooth Motion Is Important

Smooth motion reduces the risk of:

- acceleration discontinuity errors
- sudden velocity jumps
- robot reflex stops
- unstable contact forces
- unwanted object slipping

This is especially important during real robot experiments because abrupt commands may trigger safety stops.

10.4 Orientation Interpolation

Orientation interpolation was performed using quaternion SLERP:

$$q(t) = \text{slerp}(q_0, q_1, \alpha)$$

SLERP avoids discontinuities and provides smooth rotational interpolation between two orientations.

Chapter 11

Force and Torque Measurements

11.1 External Wrench Estimate

The FR3 provides an estimated external wrench:

```
state.O_F_ext_hat_K
```

This contains:

```
Fx, Fy, Fz, Mx, My, Mz
```

where:

```
Fx, Fy, Fz = external forces in newtons
```

```
Mx, My, Mz = external moments in newton-meters
```

The wrench is useful for analyzing contact with the environment.

11.2 External Joint Torque Estimate

The FR3 also provides external joint torque estimates:

```
state.tau_ext_hat_filtered
```

These values represent estimated external torques acting on the joints after filtering.

11.3 Why Force/Torque Logging Is Important

Force and torque logging allows the experimenter to answer questions such as:

- Did the robot actually touch the surface?
- Was the measured normal force close to the target?
- Did force increase during contact?
- Did the robot experience unexpected disturbances?
- Did joint torques remain within safe values?

Without logging, it is difficult to evaluate whether a force-control algorithm worked correctly.

Chapter 12

Implemented Force-Control Methods

Three force-control methods were considered:

1. Impedance control
2. Admittance control
3. Hybrid force/position control

Each method handles the relationship between motion and force differently.

12.1 Impedance Control

12.1.1 Concept

Impedance control defines a virtual dynamic relationship between position error and interaction force. The robot behaves like a virtual mechanical system, usually modeled as a mass-spring-damper.

The general impedance equation is:

$$M\ddot{x} + D\dot{x} + Kx = F$$

where:

M = virtual inertia
D = virtual damping
K = virtual stiffness
x = displacement or pose error
F = interaction force

A simplified Cartesian impedance law is:

$$F = -K(x - x_d) - D\dot{x}$$

where x_d is the desired Cartesian pose.

12.1.2 Torque Command

The desired Cartesian wrench is mapped to joint torques using the Jacobian transpose:

$$\tau = J(q)^T F + C(q, \dot{q})$$

where:

$J(q)$ = robot Jacobian
 F = desired Cartesian wrench
 $C(q, \dot{q})$ = Coriolis compensation
 τ = joint torque command

12.1.3 Practical Meaning

Impedance control makes the robot behave as if there is a virtual spring and damper between the current pose and the desired pose.

Interpretation:

High stiffness K -> the robot behaves more rigidly.

Low stiffness K -> the robot behaves more softly.

High damping D -> oscillations are reduced.

Low damping D -> the robot may become more responsive but less stable.

The FR3 is well suited to impedance control because it is torque-controlled and provides direct torque-level access through libfranka.

12.2 Admittance Control

12.2.1 Concept

Admittance control converts measured external force into desired motion.

The admittance equation is:

$$M\ddot{x} + D\dot{x} + Kx = F_{ext}$$

Solving for acceleration:

$$\ddot{x} = \frac{F_{ext} - D\dot{x} - Kx}{M}$$

Then the virtual displacement is integrated:

$$\dot{x}_{new} = \dot{x} + \ddot{x}dt$$

$$x_{new} = x + \dot{x}_{new}dt$$

The commanded pose becomes:

$$p_{cmd} = p_{nominal} + x_{admittance}$$

12.2.2 Practical Meaning

In admittance control, the robot senses a force and moves in response to that force.

Interpretation:

A softer admittance produces larger displacement for the same external force.

A harder admittance produces smaller displacement for the same external force.

Admittance control is common when the underlying robot controller is position-based. However, it can also be implemented on a torque-capable robot such as the FR3.

12.3 Hybrid Force/Position Control

12.3.1 Concept

Hybrid force/position control separates the task into directions controlled by position and directions controlled by force.

For example, when sliding an object on a table:

```
x axis -> position control
y axis -> position control
z axis -> force control
orientation -> position control
```

This is logical because the task requires the robot to move along the surface while regulating the normal force against the surface.

12.3.2 Cartesian Wrench

The Cartesian wrench is:

$$W = \begin{bmatrix} F_x \\ F_y \\ F_z \\ M_x \\ M_y \\ M_z \end{bmatrix}$$

For x and y position control:

$$F_x = -K_{xy}(x - x_d) - D_{xy}v_x$$

$$F_y = -K_{xy}(y - y_d) - D_{xy}v_y$$

For z force control:

$$F_{z,cmd} = s_f(g_{ff}F_{z,target} + K_f(F_{z,target} - F_{z,contact})) - D_f v_z$$

The final torque command is:

$$\tau = J(q)^T W + C(q, \dot{q})$$

12.3.3 Why Hybrid Control Was Needed

The final dice surface transfer task required the robot to do two things at the same time:

1. Move the dice from the start x/y position to the target x/y position.
2. Keep the dice pressed against the table with a controlled vertical force.

Pure position control could move the dice but would not regulate force. Pure force control could regulate contact but would not move the dice precisely. Hybrid control combines both requirements.

Chapter 13

Force Bias Compensation

13.1 Why Bias Compensation Is Needed

The force measurement may contain bias. This means that the measured force may not be exactly zero even when there is no intended contact force.

Bias can come from:

- tool weight
- gripper configuration
- calibration offsets
- model errors
- sensor noise
- static load effects

If the controller uses biased force values directly, it may apply too much or too little force.

13.2 Bias Estimation

At the beginning of the slide phase, the controller averages the raw force:

$$F_{z,bias} = \frac{1}{N} \sum_{i=1}^N F_{z,raw,i}$$

Then it computes the force difference:

$$F_{z,\Delta} = F_{z,raw} - F_{z,bias}$$

and the contact force:

$$F_{z,contact} = s_c F_{z,\Delta}$$

where s_c is the contact measurement sign.

13.3 Force Sign Used in the Experiments

In the experiments:

```
contact_measure_sign = -1.0;
```

This was used because downward/table contact often made:

`Fz_raw - Fz_bias`

negative.

Multiplying by `-1.0` made the contact force positive when the dice was pressing against the table.

Chapter 14

Dice Surface Transfer Experiment

14.1 Experiment Objective

The specific experiment demonstrates the complete force-control pipeline on the real robot.

The task is:

The FR3 grasps a dice from a table and slides it to another position while maintaining vertical contact force.

This experiment combines:

- real robot communication
- gripper control
- Cartesian motion
- force/torque sensing
- force bias compensation
- hybrid force/position control
- safety constraints
- CSV logging
- post-experiment analysis

14.2 Task Sequence

The sequence was:

1. Open gripper.
2. Move to pregrasp pose.
3. Move to grasp pose.
4. Close gripper and confirm grasp.
5. Start surface transfer phase.
6. Move in x/y while regulating z force.
7. Stop motion at target.
8. Open gripper.
9. Save CSV log.
10. Analyze results.

14.3 Important Experiment Values

Dice reference:

```
x = 0.45 m
y = 0.00 m
table reference z = -0.13 m
```

Calibrated end-effector values:

```
pregrasp_ee_z = 0.065;
grasp_ee_z = 0.040;
```

Object and gripper parameters:

```
dice_width_m = 0.040;
gripper_force_N = 25.0;
```

Transfer target:

```
target_x = 0.35;
target_y = 0.00;
```

Force target:

```
Fz_target = 0.8 N
```

14.4 Why Calibrated End-Effector z Values Were Used

The physical table height and the robot end-effector command height are not the same concept.

The robot commands the end-effector frame, not the bottom of the fingers or the bottom of the dice. Therefore, the correct z command must account for:

- end-effector frame location
- gripper geometry
- object size
- table height
- calibration error

For this reason, the experiment used calibrated values:

```
pregrasp_ee_z = 0.065
grasp_ee_z = 0.040
```

rather than simply commanding the physical table z value.

Chapter 15

Additional ROS 2 and MoveIt Debugging Notes

15.1 RViz and MoveIt Debugging

A significant part of the implementation involved debugging RViz and MoveIt. The main observed problem was that RViz opened, but the MotionPlanning panel did not show the expected planning group. Normally, the planning group should include `fr3_arm`, and possibly also `fr3_hand`. If the planning group is empty, RViz cannot perform motion planning correctly because it does not know which semantic group of the robot should be used.

This produced errors such as planning failure, plan-and-execute abortion, and movement timeout. These errors did not necessarily mean that the complete ROS system was broken. They indicated that MoveIt could not complete planning or execution, often because the RViz MotionPlanning panel did not have the correct semantic robot model or planning group.

The debugging process showed that the `/move_group` node had the semantic robot description as a parameter, but RViz was not receiving the semantic description through the expected topic. The missing topic was `/robot_description_semantic`. Without this topic, RViz could not parse the SRDF correctly, and the MotionPlanning panel could not load the planning groups.

The practical solution was to create a small semantic relay node. This node reads the `robot_description` and `robot_description_semantic` parameters from `/move_group` and republishes them as the topics `/robot_description` and `/robot_description_semantic`. The relay uses transient local QoS, so RViz can receive the robot description even if RViz starts after MoveIt.

After running this relay, the topics were checked again. Once `/robot_description` and `/robot_description_semantic` were available, RViz was reopened using the MoveIt RViz configuration. Then the MotionPlanning panel could load the semantic model, and the planning group `fr3_arm` became available.

This debugging step was important because it showed that a MoveIt problem may not always come from the planner itself. Sometimes the planning backend exists and the controllers are active, but RViz cannot access the semantic model. In that case, the failure appears as a planning problem, while the real cause is a missing semantic description topic.

15.2 Custom ROS 2 Package Structure

A custom ROS 2 package was also defined for node-based force-control experiments. The purpose of this package was to organize force-control functionality into reusable ROS nodes. This modular approach allows different parts of the system to be tested independently.

The package included nodes for publishing test wrenches, estimating external wrenches, and implementing admittance control. This structure is useful because force-control systems often require several separate components: one component may estimate force, another may filter or process it, and another may generate motion commands based on the measured interaction.

The advantage of this architecture is modularity. A wrench estimator can be tested independently from the controller. A test wrench publisher can be used to validate the communication pipeline before connecting the system to real sensor data. An admittance controller node can then use the measured or estimated wrench to update the desired robot motion.

Although the final low-level experiments were mainly performed through direct `libfranka` programs, the ROS package structure remains useful for future extensions. For example, vision-based object localization, external sensors, or high-level task planning could be integrated more easily through ROS 2 nodes.

Chapter 16

Practical Commands Used During the Experimental Workflow

This subsection presents the practical terminal commands that were used during the development and execution of the FR3 experiments. The commands are organized according to the usual order followed during the experiments. This order is important because the real robot should not be controlled before the network connection, the Desk state, the Franka Control Interface, and the software environment have been verified.

16.1 Basic ROS 2 and Workspace Setup

Before running any ROS 2, MoveIt, or Gazebo command, the ROS 2 environment and the Franka workspace must be sourced. This ensures that the terminal can find the correct ROS 2 packages, launch files, controllers, robot descriptions, and MoveIt configurations.

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash
```

To verify that the Franka packages are visible in the current terminal, the following command can be used:

```
ros2 pkg list | grep franka
```

A correct installation should show packages such as `franka_bringup`, `franka_description`, `franka_fr3_moveit_config`, `franka_hardware`, and other related Franka packages. If only a small number of Franka packages appear, then the workspace has either not been built correctly or has not been sourced correctly.

16.2 Ethernet Configuration for the FR3 Controller

Before communicating with the real FR3, the Ethernet interface of the workstation must be configured. In the experiments, the Ethernet interface was usually named `enp3s0`. The workstation was configured with the static IP address `172.16.0.1/24`, while the FR3 controller was accessed at `172.16.0.2`.

A manual configuration can be performed with:

```
sudo nmcli dev set enp3s0 managed no
```

```
sudo ip addr flush dev enp3s0
sudo ip route flush dev enp3s0
sudo ip link set enp3s0 up
sudo ip addr add 172.16.0.1/24 dev enp3s0
sudo ip route replace 172.16.0.0/24 dev enp3s0 src 172.16.0.1
sudo ip route flush cache
```

After this, the IP address can be checked using:

```
ip addr show enp3s0
```

The expected result is that the interface contains:

```
inet 172.16.0.1/24
```

Then the route to the controller can be checked:

```
ip route get 172.16.0.2
```

A correct route should show that the packets go through **enp3s0** with source address **172.16.0.1**.

Finally, the controller connection is checked using:

```
ping -c 10 172.16.0.2
```

The expected result is:

```
0% packet loss
```

If there is packet loss, the robot should not be controlled. The Ethernet cable, IP configuration, controller power, and physical connection must be checked first.

16.3 Alternative Network Configuration Using nmcli

An alternative way to configure the Ethernet interface is to create a NetworkManager connection. This is useful when the same configuration must be reused multiple times.

```
sudo nmcli con delete fr3-fci 2>/dev/null
```

```
sudo nmcli con add type ethernet ifname enp3s0 con-name fr3-fci \
  ipv4.method manual \
  ipv4.addresses 172.16.0.1/24 \
  ipv6.method ignore
```

```
sudo nmcli con up fr3-fci
```

After activating the connection, the configuration is checked again:

```
ip addr show enp3s0
ip route get 172.16.0.2
ping -c 10 172.16.0.2
```

This method is cleaner when the same laptop is used repeatedly for the FR3 experiments.

16.4 Desk Preparation Before Real Robot Control

After the network connection is working, the robot must be prepared through Desk. Desk is opened in the browser using:

```
https://172.16.0.2
```

In some setups, when the laptop is connected directly to the robot-side network, Desk may be accessed through:

```
https://192.168.0.1
```

However, for the main experiments the controller network at 172.16.0.2 was used.

Before running any real robot program, the following actions must be completed in Desk:

1. Unlock the robot joints and brakes.
2. Clear errors if the robot is in an error or reflex state.
3. Put the robot in the correct operation mode.
4. Activate the Franka Control Interface.
5. Verify that the robot is ready and operational.
6. Keep the emergency stop accessible.

This step is not optional. Even if the network ping works, the robot cannot be safely controlled unless Desk shows that the robot is ready and FCI is active.

16.5 Testing libfranka Communication

After Desk preparation and FCI activation, the next validation step is to run `echo_robot_state`. This confirms that `libfranka` can receive real-time robot-state packets.

Depending on the installation, the command can be:

```
/home/danai/franka_ros2_ws/install/libfranka/bin/echo_robot_state 172.16.0.2
```

or:

```
/home/danai/franka_ros2_ws/build/libfranka/examples/echo_robot_state 172.16.0.2
```

If the robot state is printed continuously, then the low-level communication is working correctly. If this command fails, no real robot motion should be attempted.

A common failure is:

```
libfranka: UDP receive: Timeout
```

This means that the computer may be able to ping the controller, but `libfranka` is not receiving the real-time packets required for control. In that case, old processes should be stopped, the robot should be recovered from Desk, and FCI should be activated again.

Useful cleanup commands are:

```

sudo pkill -f fr3_pick_place_ball
sudo pkill -f fr3_pick_direct
sudo pkill -f fr3_admittance_task_test
sudo pkill -f fr3_impedance_task_test
sudo pkill -f fr3_hybrid_force_position_test
sudo pkill -f echo_robot_state

```

After this cleanup, `echo_robot_state` should be tested again.

16.6 Building Direct libfranka C++ Programs

The direct libfranka programs were built separately from ROS. A typical build process for a standalone program is:

```

cd ~/Desktop/scripts_no_ros/program_folder
mkdir -p build
cd build

cmake .. \
  -DCMAKE_PREFIX_PATH=$HOME/franka_ros2_ws/install/libfranka \
  -DCMAKE_BUILD_TYPE=Release

make -j

```

The `Release` build type is important because the real-time control loop should run efficiently. Debug builds may be too slow for stable control.

16.7 Running Direct libfranka Programs

Direct libfranka programs were executed using `sudo` and an explicit `LD_LIBRARY_PATH`. This ensures that the correct libfranka shared library is used.

The general command format is:

```

sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./program_name 172.16.0.2

```

For example, the force-reading program can be run as:

```

cd ~/Desktop/scripts_no_ros/fr3_read_forces/build

sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_read_forces 172.16.0.2

```

The impedance controller can be run in soft mode:

```

cd ~/Desktop/scripts_no_ros/fr3_impedance_task_test/build

sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_impedance_task_test 172.16.0.2 soft

```


or in hard mode:

```
cd ~/Desktop/scripts_no_ros/fr3_impedance_task_test/build
```

```
sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_impedance_task_test 172.16.0.2 hard
```

The hybrid force/position controller can be run in a low-gain configuration:

```
cd ~/Desktop/scripts_no_ros/fr3_hybrid_force_position_test/build
```

```
sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_hybrid_force_position_test 172.16.0.2 low
```

or in a higher-gain configuration:

```
cd ~/Desktop/scripts_no_ros/fr3_hybrid_force_position_test/build
```

```
sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_hybrid_force_position_test 172.16.0.2 high
```

The same structure was used for other standalone programs such as pick-and-place, Cartesian motion, admittance control, and the final dice surface-transfer experiment.

16.8 Clean Build of the ROS 2 Franka Workspace

If the ROS 2 workspace has build problems, especially due to conflicting versions of `libfranka`, a clean build may be required. The workspace is cleaned and rebuilt with:

```
cd ~/franka_ros2_ws
source /opt/ros/humble/setup.bash

rm -rf build install log

colcon build --symlink-install \
  --event-handlers console_direct+ \
  --cmake-args \
    -DCMAKE_BUILD_TYPE=Release \
    -DBUILD_TESTS=OFF \
    -DBUILD_TESTING=OFF \
    -DBUILD_EXAMPLES=ON
```

After the build finishes, the workspace must be sourced:

```
source ~/franka_ros2_ws/install/setup.bash
```

Then the Franka packages are checked:

```
ros2 pkg list | grep franka
```

This build step is not required before every experiment, but it is required after modifying packages, changing dependencies, or fixing build/configuration problems.

16.9 Running MoveIt with Fake Hardware

The first ROS 2 test should be done with fake hardware. This does not move the real robot. It only moves the simulated robot state in RViz.

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

ros2 launch franka_fr3_moveit_config moveit.launch.py \
  robot_ip:=dont-care \
  use_fake_hardware:=true \
  robot_type:=fr3 \
  load_gripper:=false \
  ee_id:=none
```

After RViz opens, the planning group should contain:

```
fr3_arm
```

In a second terminal, the system can be checked using:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

ros2 node list
ros2 control list_controllers
ros2 topic echo /joint_states --once
```

The expected result is that `/move_group` exists, the controllers are active, and `/joint_states` contains the FR3 joints.

16.10 Checking MoveIt Planning Groups

If the RViz MotionPlanning panel does not show the planning group, the semantic model should be checked.

First, check whether `/move_group` has the semantic description:

```
ros2 param get /move_group robot_description_semantic | grep "group name"
```

The expected result is:

```
<group name="fr3_arm">
<group name="fr3_hand">
```

Then check whether the robot description topics exist:

```
ros2 topic list | grep robot_description
```

The expected topics are:

```
/robot_description
/robot_description_semantic
```

If `/robot_description_semantic` is missing, RViz may fail to load the planning groups.

16.11 Temporary RViz Semantic Relay Fix

If RViz does not receive `/robot_description_semantic`, a temporary relay node can be created. This node reads the robot descriptions from `/move_group` and republishes them as topics.

The script can be created with:

```
cat > /tmp/publish_fr3_semantic.py <<'EOF'
import rclpy
from rclpy.node import Node
from rcl_interfaces.srv import GetParameters
from std_msgs.msg import String
from rclpy.qos import QoSProfile, QoSDurabilityPolicy, QoSReliabilityPolicy

class MoveItDescriptionRelay(Node):
    def __init__(self):
        super().__init__('moveit_description_relay')

        qos = QoSProfile(depth=1)
        qos.durability = QoSDurabilityPolicy.TRANSIENT_LOCAL
        qos.reliability = QoSReliabilityPolicy.RELIABLE

        self.pub_robot = self.create_publisher(String, '/robot_description', qos)
        self.pub_semantic = self.create_publisher(String, '/robot_description_semantic', qos)

        self.client = self.create_client(GetParameters, '/move_group/get_parameters')

        self.get_logger().info('Waiting for /move_group/get_parameters...')
        if not self.client.wait_for_service(timeout_sec=10.0):
            raise RuntimeError('Could not find /move_group/get_parameters service')

        req = GetParameters.Request()
        req.names = ['robot_description', 'robot_description_semantic']

        future = self.client.call_async(req)
        rclpy.spin_until_future_complete(self, future)

        result = future.result()
        if result is None:
            raise RuntimeError('Failed to read parameters from /move_group')

        self.robot_description = result.values[0].string_value
        self.robot_description_semantic = result.values[1].string_value

        if not self.robot_description.strip():
            raise RuntimeError('robot_description is empty')

        if not self.robot_description_semantic.strip():
```

```

        raise RuntimeError('robot_description_semantic is empty')

    self.timer = self.create_timer(1.0, self.publish_descriptions)

    def publish_descriptions(self):
        robot_msg = String()
        robot_msg.data = self.robot_description
        self.pub_robot.publish(robot_msg)

        semantic_msg = String()
        semantic_msg.data = self.robot_description_semantic
        self.pub_semantic.publish(semantic_msg)

def main():
    rclpy.init()
    node = MoveItDescriptionRelay()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
EOF

```

Then it is executed with:

```

source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

python3 /tmp/publish_fr3_semantic.py

```

This terminal should remain open. In another terminal, verify that the topics exist:

```
ros2 topic list | grep robot_description
```

Then RViz can be reopened:

```
pkill -f rviz2
```

```
rviz2 -d $(ros2 pkg prefix franka_fr3_moveit_config)/share/franka_fr3_moveit_config/rviz2
```

After this, the MotionPlanning panel should show the planning group `fr3_arm`.

16.12 Running Gazebo Simulation

Gazebo simulation should be tested before real robot execution when changes are made to robot description files, launch files, controllers, or object placement.

First, the available Gazebo launch files can be listed:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

ls $(ros2 pkg prefix franka_gazebo_bringup)/share/franka_gazebo_bringup/launch
```

Then an available Gazebo launch file can be executed. For example:

```
ros2 launch franka_gazebo_bringup gazebo.launch.py
```

or, depending on the available files:

```
ros2 launch franka_gazebo_bringup fr3.launch.py
```

In a second terminal, the controllers and joint states are checked:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

ros2 control list_controllers
ros2 topic echo /joint_states --once
```

If joint states are published and the controllers are active, the simulation is connected correctly to the ROS 2 control system.

16.13 Running the Real FR3 with MoveIt

The real FR3 should only be launched with MoveIt after all previous validation steps have succeeded. This means that ping works, `echo_robot_state` works, Desk is ready, FCI is active, and the workspace is sourced correctly.

The real hardware MoveIt launch is:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash

ros2 launch franka_fr3_moveit_config moveit.launch.py \
  robot_ip:=172.16.0.2 \
  use_fake_hardware:=false \
  robot_type:=fr3 \
  load_gripper:=false \
  ee_id:=none
```

In a second terminal:

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash
```

```
ros2 control list_controllers
ros2 topic echo /joint_states --once
```

In RViz, the first real movement should always be very small. The operator should first press **Plan**, inspect the trajectory, and only then press **Execute**. The emergency stop should always be close.

16.14 Running Real FR3 Example Controllers

The Franka ROS 2 stack also provides example controllers. These should be tested carefully and in a safe order.

A state-only bringup, without commanding movement, can be launched with:

```
ros2 launch franka_bringup franka.launch.py \
  robot_ip:=172.16.0.2 \
  robot_type:=fr3 \
  load_gripper:=false \
  use_fake_hardware:=false
```

A simple joint position example can be launched with:

```
ros2 launch franka_bringup example.launch.py \
  robot_ips:=172.16.0.2 \
  controller_names:=joint_position_example_controller
```

A model example can be launched with:

```
ros2 launch franka_bringup example.launch.py \
  robot_ips:=172.16.0.2 \
  controller_names:=model_example_controller
```

A Cartesian pose example can be launched with:

```
ros2 launch franka_bringup example.launch.py \
  robot_ips:=172.16.0.2 \
  controller_names:=cartesian_pose_example_controller
```

A Cartesian orientation example can be launched with:

```
ros2 launch franka_bringup example.launch.py \
  robot_ips:=172.16.0.2 \
  controller_names:=cartesian_orientation_example_controller
```

The recommended testing order is:

1. joint_position_example_controller
2. model_example_controller

3. cartesian_pose_example_controller
4. cartesian_orientation_example_controller
5. joint_impedance_example_controller

Velocity-based controllers should not be tested first because they can produce more dynamic motion. These include:

```
joint_velocity_example_controller
cartesian_velocity_example_controller
mobile_cartesian_velocity_example_controller
```

Before each new controller, the previous launch should be stopped with **Ctrl+C**, the robot should be allowed to stop completely, and Desk should be checked for errors.

16.15 Monitoring During ROS Experiments

During ROS 2 experiments, a second terminal should always be used for monitoring. The active controllers can be checked with:

```
ros2 control list_controllers
```

The joint state can be checked with:

```
ros2 topic echo /joint_states --once
```

To verify that a specific controller is active:

```
ros2 control list_controllers | grep active
```

The running ROS nodes can be checked with:

```
ros2 node list
```

These monitoring commands are useful because they confirm whether the controller was loaded, whether the robot state is being published, and whether the ROS system is still active.

16.16 Final Command Sequence Before a Real Force-Control Experiment

Before running a real direct `libfranka` force-control experiment, the complete practical sequence is:

```

# 1. Configure Ethernet
sudo nmcli dev set enp3s0 managed no
sudo ip addr flush dev enp3s0
sudo ip route flush dev enp3s0
sudo ip link set enp3s0 up
sudo ip addr add 172.16.0.1/24 dev enp3s0
sudo ip route replace 172.16.0.0/24 dev enp3s0 src 172.16.0.1
sudo ip route flush cache

# 2. Check connection
ping -c 10 172.16.0.2

# 3. Open Desk in browser
# https://172.16.0.2
# Unlock robot, clear errors, activate FCI

# 4. Check libfranka communication
/home/danai/franka_ros2_ws/install/libfranka/bin/echo_robot_state 172.16.0.2

# 5. Run the desired direct-control program
cd ~/Desktop/scripts_no_ros/fr3_hybrid_force_position_test/build

sudo env LD_LIBRARY_PATH=$HOME/franka_ros2_ws/install/libfranka/lib:$LD_LIBRARY_PATH \
./fr3_hybrid_force_position_test 172.16.0.2 low

```

This command sequence represents the minimum safe procedure before running a real force-control test. If any step fails, the experiment should not continue until the issue is fixed.

16.17 Summary of When Each Command Group Is Used

The command groups can be summarized as follows:

- The Ethernet configuration commands are used before any real robot communication.
- The ping command is used to verify basic network connectivity.
- The Desk preparation is used to unlock the robot and activate FCI.
- The `echo_robot_state` command is used to verify `libfranka` communication.
- The direct `libfranka` run commands are used for low-level force-control experiments.
- The ROS 2 source commands are used before every ROS, MoveIt, or Gazebo command.
- The fake hardware launch is used before testing real robot planning.
- The Gazebo launch is used for simulation testing.
- The real MoveIt launch is used only after all safety checks pass.

- The monitoring commands are used during ROS experiments to verify controllers and joint states.

This command workflow makes the experimental process systematic and reproducible. It also reduces the probability of unsafe robot behavior, because every real robot execution is preceded by network, Desk, FCI, and communication validation.

16.18 Final Reproducible Workflow

The final reproducible workflow used during the project can be summarized as follows:

1. Clean old ROS and robot-control processes.
2. Source ROS 2 and the Franka workspace.
3. Check that all Franka packages are visible.
4. Test MoveIt with fake hardware.
5. Verify that `/move_group`, the controllers, and `/joint_states` exist.
6. Fix RViz semantic loading if the planning group is empty.
7. Test Gazebo simulation if required.
8. Configure the Ethernet interface for the FR3 controller.
9. Verify ping with zero packet loss.
10. Open Desk, unlock the robot, clear errors, and activate FCI.
11. Verify `libfranka` communication with `echo_robot_state`.
12. Launch MoveIt with real hardware only after all previous steps succeed.
13. Execute only a very small first motion.
14. Run example controllers only in a safe order.
15. Run direct `libfranka` force-control programs for the final experiments.
16. Log all experiment data to CSV.
17. Analyze position, force, and torque plots after execution.

This workflow provides a systematic development path from simulation to real hardware. It also supports the main objective of this thesis: implementing and evaluating force-control strategies on the FR3 in a safe, reproducible, and experimentally verifiable way.

Chapter 17

Troubleshooting Guide

17.1 Desk Opens but the Arm Is Disconnected

Symptoms:

Arm disconnected
Safety booting...
No Arm Connected
Robot not operational
PROG mode disabled

Meaning:

The controller is reachable, but the controller cannot communicate with the robot arm.

Possible causes:

- loose robot-controller Ethernet cable
- disconnected robot-controller cable
- emergency stop active
- safety chain triggered
- arm initialization failure

Important root cause found during debugging:

The Ethernet cable connecting the robot to the controller was loose or improperly connected.

This caused:

- Arm disconnected
- Safety booting...
- No Arm Connected
- 192.168.0.1 unreachable
- disabled PROG mode
- failed self tests

17.2 Recovery Procedure for Arm Disconnection

Recommended recovery:

1. Power off controller and robot.
2. Wait about 1 minute.
3. Reconnect the robot-controller Ethernet cable firmly on both sides.
4. Release the emergency stop.
5. Power on the controller.
6. Wait about 2 minutes.
7. Power on the robot.
8. Wait 3--5 minutes for initialization.
9. Open Desk at <https://172.16.0.2>.
10. Verify that the robot is operational.

Expected healthy state:

Robot operational
Arm connected
PROG mode enabled
self tests functional
X5 network active
192.168.0.1 reachable

17.3 Link detected: no

Check:

```
sudo ethtool enp3s0
```

If the result is:

Link detected: no

then the problem is physical.

Fix:

- check Ethernet cable
- reconnect RJ45
- use correct port
- verify controller power
- check Ethernet LEDs

17.4 libfranka: UDP receive: Timeout

Meaning:

The PC may ping the robot/controller, but libfranka is not receiving robot-state packets.

Check:

- Is FCI activated?
- Is the robot recovered in Desk?
- Is another process already connected?
- Is the firewall blocking communication?
- Is the route correct?
- Is the Ethernet connection stable?

Useful commands:

```
sudo pkill -f fr3_pick_place_ball
sudo pkill -f fr3_pick_direct
sudo pkill -f fr3_admittance_task_test
sudo pkill -f fr3_impedance_task_test
sudo pkill -f echo_robot_state
```

Then recover the robot from Desk and try again.

17.5 Desk Reachable but 192.168.0.1 Unreachable

This can happen when:

- the controller network is working,
- but the robot/X5 network is not active because the arm is not initialized.

Fix the arm connection first. Do not focus only on Ubuntu networking.

17.6 X3.1 Triggered

This usually indicates a safety-chain or arm-communication issue.

Check:

- emergency stop
- robot-controller cable
- arm connection
- safety boot state
- Desk safety messages

17.7 Self Test Fails With Internal Error

If the arm is disconnected or safety boot is incomplete, the self test may fail.

This is usually a consequence of the arm communication failure, not the original root cause.

17.8 WATCHMAN Shows Validated but Robot Is Not Operational

The WATCHMAN page may show:

Validated

This only means that the safety configuration is valid.

It does **not** necessarily mean that the robot arm is connected or operational.

17.9 Login Mode

Certain operations require login as:

fr3-safety

instead of:

fr3-operator

17.10 Gripper Does Not Confirm Grasp

Possible causes:

- wrong object width
- gripper not centered on object
- grasp height too high
- grasp height too low
- insufficient gripper force
- object slipped
- epsilon values too strict

Fixes:

- adjust `dice_width_m`
- adjust `gripper_force_N`
- recalibrate `grasp_ee_z`
- check object placement
- reduce approach speed

17.11 Force Does Not Increase

Possible causes:

- robot is not actually in contact
- force sign is wrong
- contact bias is wrong
- object is not touching the table
- force target is too low

Fixes:

- verify contact visually
- check `Fz_raw`, `Fz_bias`, and `Fz_delta`
- test `contact_measure_sign` = -1.0 or +1.0
- lower the grasp/contact height carefully

17.12 Force Is Too High or Unstable

Possible causes:

- force gain too high
- damping too low
- contact surface too stiff
- z safety wall interfering
- measurement bias incorrect
- motion too fast

Fixes:

- reduce force-control gain
- increase damping
- reduce force target
- slow the slide
- improve bias averaging

17.13 Final Position Error Is Too Large

Possible causes:

- x/y stiffness too low
- slide too fast
- gripper-object slip
- table friction
- target calibration error
- force controller interfering with planar tracking

Fixes:

- increase x/y stiffness carefully
- slow the movement
- improve grasp stability
- compensate final target
- tune hybrid controller gains